

딥러닝

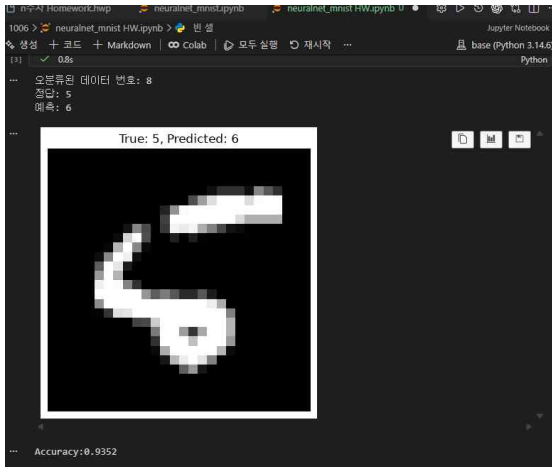
6주차 Homework



충북대학교
CHUNGBUK NATIONAL UNIVERSITY

날짜	2026.10.06.
이름	권택주
학번	2023042028
학과	지능로봇공학과
담당 교수	전명근 교수님

1. 오류 결과 영상



2. 수정된 코드 동작 설명

전체 코드	함수 정의
<pre>import sys, os, pickle sys.path.append(os.pardir) # Add parent directory to the path import numpy as np import matplotlib.pyplot as plt from dataset.mnist import load_mnist from PIL import Image x, t = get_data() network = init_network() accuracy_cnt = 0 first_error_shown = False for i in range(len(x)): y = predict(network, x[i]) p = np.argmax(y) # Get the index of the highest probability if p == t[i]: accuracy_cnt += 1 elif not first_error_shown: print("오분류된 데이터 번호:", i) print("정답:", t[i]) print("예측:", p) img = x[i].reshape(28, 28) plt.imshow(img, cmap='gray') plt.title(f"True: {t[i]}, Predicted: {p}") plt.axis('off') plt.show() first_error_shown = True print("Accuracy:" + str(float(accuracy_cnt) / len(x)))</pre>	<pre>def get_data(): (x_train, t_train), (x_test, t_test) = load_mnist(normalize=True, flatten=True, one_hot_label=False) return x_test, t_test def init_network(): with open("dataset/sample_weight.pkl", 'rb') as f: network = pickle.load(f) return network def sigmoid(x): return 1 / (1 + np.exp(-x)) def softmax(a): c = np.max(a) exp_a = np.exp(a - c) # Prevent overflow by subtracting the max sum_exp_a = np.sum(exp_a) y = exp_a / sum_exp_a return y def predict(network, x): W1, W2, W3 = network['W1'], network['W2'], network['W3'] b1, b2, b3 = network['b1'], network['b2'], network['b3'] a1 = np.dot(x, W1) + b1 z1 = sigmoid(a1) a2 = np.dot(z1, W2) + b2 z2 = sigmoid(a2) a3 = np.dot(z2, W3) + b3 y = softmax(a3) return y</pre>

[코드 실행 순서]

1. load_mnist()를 이용하여 MNIST 테스트 데이터 10,000개를
2. sample_weight.pkl에 저장된 학습된 신경망의 가중치와 편향을 불러옵니다.
3. 각 테스트 이미지를 신경망에 입력
4. 입력 데이터는 첫 번째 은닉층과 두 번째 은닉층에서 sigmoid 활성화 함수 통과
5. 출력층에서는 softmax 함수를 사용하여 숫자 0~9 각각에 대한 확률 계산
6. np.argmax(y)를 이용해 가장 높은 확률을 가진 숫자를 예측 결과로 결정
7. 예측값 p와 실제 정답 t[i]를 비교
8. 처음으로 $p \neq t[i]$ 인 데이터가 발견되면 해당 MNIST 이미지를 28×28 형태로 변환하여 화면에 출력합니다.
9. 이미지 위에 실제 정답과 신경망의 예측 결과 표시
10. 첫 오분류 이후에도 전체 테스트 데이터의 분류를 계속하여 최종 정확도 계산

[first_error_shown]

- 첫 번째 오분류 이미지만 1번 출력하기 위한 상태 변수
- 테스트 중 처음 발견되는 오분류 이미지만 화면에 출력하기 위해 사용. 초기값을 False로 설정해 첫 오분류가 발생하면 이미지를 출력한 뒤, True로 변경.
- 이후 발생하는 오분류는 출력하지 않으며, 전체 테스트 데이터에 대한 정확도 계산 수행